FINAL-YEAR PROJECT — CORRECTED & STRUCTURED EDITION (v2.0)

AI-Powered Smart Water Management System with Time-to-Depletion Prediction and Automated Flow Control

A Complete Project Blueprint, Methodology & Implementation Guide
Integrating IoT • Cloud Computing • Machine Learning • Embedded Control

Submitted by: Trijal Burman Gayatri Apotikar Prashant Bajpai Hardik Dhalla

Department: Electronics & Tellecommunication Engineering | **Institution:** Symbiosis Institute of Technology

Project Guide: Dr. D Marshiana | **Academic Year:** 2025–2026

Table of Contents

Chapter 1 — Abstract

Chapter 2 — Introduction & Problem Statement

Chapter 3 — Objectives & Scope

Chapter 4 — Technology Stack & Justification

Chapter 5 — System Architecture

Chapter 6 — Hardware Design & Physical Prototype

Chapter 7 — Software Architecture & Deployment

Chapter 8 — Machine Learning Methodology

Chapter 9 — Control Strategy, Safety & Security

Chapter 10 — Implementation Methodology (Phase Plan)

Chapter 11 — Testing & Validation Plan

Chapter 12 — Dashboard & Visualization Design

Chapter 13 — Project Timeline

Chapter 14 — Challenges & Mitigations

Chapter 15 — Expected Outcomes & Conclusion

Chapter 16 — Future Enhancements

Appendix A — Professional Phrasing Guide

Appendix B — MVP vs. Advanced Scope

Chapter 1 — Abstract

Water storage systems in hostels, apartments, and institutions often suffer from sudden depletion because users and administrators know only the *current* water level, not the *future availability* of water. This project presents an AI-powered smart water management system that predicts the time-to-depletion of a water tank and automatically controls water usage to prevent complete exhaustion.

The system uses an ESP32 microcontroller with an ultrasonic sensor for water-level measurement and a flow sensor for consumption monitoring. Sensor data is transmitted via MQTT to a cloud server, stored in InfluxDB, visualized using Grafana, and processed by a machine-learning model that predicts future water demand from historical consumption patterns. When the predicted depletion time falls below a safety threshold, the system issues control commands to the ESP32 to adjust a motorized valve and extend the available water supply. Local fallback logic ensures safe operation even during network or cloud failures.

The proposed system integrates IoT, cloud computing, time-series analytics, machine learning, and embedded control into a single practical solution for smart water management.

Chapter 2 — Introduction & Problem Statement

2.1 The Problem

Consider a hostel or apartment building. During summer, the municipality suddenly cuts the water supply. Residents continue consuming water normally, the tank silently drains, and only **after** it becomes empty does management call a water tanker. Everyone suffers in the interim.

2.2 Limitations of Existing Systems

- Conventional indicators show only the current level (e.g., “Tank = 40%”).
- They provide no prediction of *when* the tank will empty.
- They take no automatic preventive action.

2.3 Proposed Solution

The system predicts depletion **before** it happens — analogous to Google Maps predicting your arrival time, but instead predicting when the tank will run dry — and then automatically rations supply to extend availability until a refill arrives.

Chapter 3 — Objectives & Scope

3.1 Primary Objectives

1. Measure tank water level using an ultrasonic sensor.
2. Measure consumption using a flow sensor.
3. Transmit telemetry to the cloud via MQTT.
4. Store time-series data in InfluxDB.
5. Predict time-to-depletion using machine learning.
6. Visualize live data and predictions in Grafana.
7. Automatically control a valve when depletion risk is high.
8. Maintain safe local operation if internet/cloud fails.

3.2 Scope

In Scope	Out of Scope (unless extended)
Single-tank level & flow monitoring; cloud logging; depletion prediction; dashboard; valve control; alerts; local failsafe	Building-wide plumbing automation; multi-tank optimization; billing integration; municipal network integration; large-scale deployment

Chapter 4 — Technology Stack & Justification

Layer	Technology	Why Chosen
Edge controller	ESP32	Dual-core, Wi-Fi built-in, low cost, ideal for IoT
Level sensing	HC-SR04 / JSN-SR04T	Non-contact, inexpensive, adequate accuracy after filtering
Flow sensing	YF-S201	Pulse-based hall-effect flow meter, simple interfacing
Actuation	Motorized ball valve + relay/driver	Supports partial opening for true flow rationing
Communication	MQTT	Lightweight, fast, minimal bandwidth — ideal for constrained devices
Database	InfluxDB	Purpose-built for high-frequency time-series sensor data
Backend	Python + FastAPI	Async ingestion, REST API, easy ML integration
Visualization	Grafana	Native InfluxDB support, graphs, gauges, alerting
Machine Learning	Scikit-learn / XGBoost	Strong tabular/time-series regression with explainable features
Deployment	Docker on AWS EC2	Portable, reproducible deployment; easy migration between servers

Chapter 5 — System Architecture

5.1 Logical Components

Component	Role	Example Output
Sensors (the eyes)	Measure level and flow	“Tank 72% full”, “18 L/min”
ESP32 (edge brain)	Read, filter, convert, publish; execute commands; local failsafe	JSON telemetry every 5–10 s
Cloud server	MQTT broker, ingestion, storage, API	Historical time-series in InfluxDB
AI model	Predict future demand → time-to-depletion	“Empty in 2 h 18 min”
Controller	Compare prediction to thresholds; issue commands	Valve set-point change

5.2 High-Level Data Flow

```
Water Tank
   |--- Ultrasonic Level Sensor
   |--- Flow Sensor
   v
ESP32 Edge Controller
   |  MQTT over Wi-Fi (TLS, authenticated)
   v
MQTT Broker (Mosquitto)
   |
   v
Cloud Server / AWS EC2 (Docker)
   |--- InfluxDB (time-series store)
   |--- FastAPI backend (ingestion + REST API)
   |--- ML Prediction Service
   |--- Grafana Dashboard
   v
Control Decision Engine
   |  MQTT command
   v
ESP32 --> Motorized Valve / Pump Controller
```

5.3 MQTT Topic & Payload Design

Topic	Direction	Purpose
`watertank/telemetry`	ESP32 → Cloud	Level, volume, flow, valve %, pump state, RSSI
`watertank/command`	Cloud → ESP32	Mode, valve set-point, reason, priority, timestamp
`watertank/status`	ESP32 → Cloud	Heartbeat, online state, local mode

```
Telemetry example:
{ "device_id":"esp32-01", "level_cm":22.4, "level_percent":68.5,
  "volume_liters":412.3, "flow_lpm":12.8, "valve_percent":100,
  "pump_state":false, "wifi_rssi":-61 }

Command example:
{ "mode":"RATION", "valve_percent":40,
  "reason":"predicted_empty_in_75_min", "priority":"high" }
```

Chapter 6 — Hardware Design & Physical Prototype

6.1 Corrected Prototype Layout (Elevated-Tank Model)

The consumer line is gravity-fed from an elevated tank; the pump is used only for **refill simulation**. This avoids running a pump against a closed valve and mimics a real overhead tank.

```
Bucket B (reservoir / consumer catch)
   |  pump (refill only)
   v
Bucket A = elevated tank  <-- ultrasonic sensor on top (inside stilling pipe)
   |  gravity outlet
   v
Flow Sensor (YF-S201)
   v
Motorized Ball Valve
   v
Consumer outlet --> Bucket B
```

6.2 Bill of Materials

Item	Qty	Notes
ESP32 DevKit	1	Main controller
HC-SR04 (or waterproof JSN-SR04T)	1	Level sensing
YF-S201 flow sensor	1	Outflow measurement
Motorized ball valve (or proportional valve)	1	Partial-flow control
Relay module / motor driver + optocoupler	1	Valve & pump actuation
5 V DC pump	1	Refill simulation only
Float switch	1	Hard-wired low-level pump cut-off
PVC pipe (stilling well), tubing, buckets	—	Prototype plumbing
5 V PSU + battery/UPS backup	1	Power-failure resilience

6.3 Critical Hardware Corrections

Correction 1 — Valve type: A standard solenoid valve is strictly ON/OFF and cannot “partially close.” Use a **motorized ball valve** (percentage control), a proportional valve, or — if only a solenoid is available — implement **timed duty-cycle rationing** (e.g., open 10 s / closed 20 s) with care to avoid water hammer.

Correction 2 — Pump protection: Add a mechanical **float switch** at the tank bottom wired to cut pump power independently of software, preventing dry-run damage and guaranteeing a safe state even if the AI or cloud fails.

Correction 3 — Ultrasonic noise: Mount the sensor over an open-ended **PVC stilling pipe** extending into the water to suppress ripple-induced noise; combine with median filtering in firmware.

6.4 Wiring Reference

Module	Connection Notes
HC-SR04	VCC→5 V, GND→GND, TRIG→GPIO; ECHO→GPIO via **voltage divider** (5 V → 3.3 V)
YF-S201	VCC→5 V, GND→GND, Signal→GPIO with pull-up; count pulses via hardware interrupt
Motorized valve	Drive via relay/motor driver with opto-isolation; separate supply if required
Pump	Relay/MOSFET with flyback diode; interlocked with float switch

6.5 Calibration Procedure

- **Level:** record sensor distance at known water heights; build a distance–volume map using tank geometry; reject out-of-range readings.
- **Flow:** collect water for a fixed time into a measuring container; compute pulses-per-liter; verify at multiple flow rates.

Chapter 7 — Software Architecture & Deployment

7.1 Firmware Responsibilities (ESP32)

1. Connect to Wi-Fi and authenticated MQTT broker.
2. Read, filter (median + moving average), and convert sensor data.
3. Publish telemetry; subscribe to command topic.
4. Actuate valve/pump; enforce local safety rules.
5. Buffer readings to flash during outages; replay on reconnect.

```
firmware/esp32/
  main.cpp | config.h
  sensors/ (ultrasonic.cpp, flow.cpp)
  actuators/ (valve.cpp, pump.cpp)
  network/ (wifi.cpp, mqtt_client.cpp)
  logic/ (edge_control.cpp, failsafe.cpp)
  utils/ (filters.cpp, calibration.cpp)
```

7.2 Cloud Services

- **Option 1 (self-hosted):** AWS EC2 running Docker containers — Mosquitto, InfluxDB, Grafana, FastAPI, ML service.
- **Option 2 (managed, less DevOps):** InfluxDB Cloud + Grafana Cloud + lightweight broker (or AWS IoT Core).

7.3 Containerized Deployment

```
services:
  mosquitto:  image: eclipse-mosquitto
  influxdb:   image: influxdb
  grafana:    image: grafana/grafana
  api:        build: ./backend
  predictor:  build: ./ml-service
```

7.4 Repository Structure

```
smart-water-system/
  firmware/esp32/
  cloud/ (docker-compose.yml, mosquitto/, influxdb/, grafana/)
  backend/ (app.py, mqtt_ingest.py, api.py)
  ml/ (train.py, predict.py, data/)
  dashboards/ (water_dashboard.json)
  docs/ (report, architecture diagrams)
  README.md
```

Chapter 8 — Machine Learning Methodology

8.1 Why Simple Division Is Not ML

`Volume ÷ current flow` is a rule-based baseline. True ML adds value by **predicting future demand** from historical patterns (morning/evening peaks, weekday vs. weekend behavior), producing a more reliable time-to-depletion estimate.

8.2 Pipeline

1. **Collect:** timestamp, level, volume, flow, valve %, pump state.
2. **Feature engineering:** hour of day, day of week, 5/15/30-min rolling flow averages, level derivative, valve position, recent peaks, current volume %.
3. **Target:** average flow rate over the next 30 minutes (recommended) or direct time-to-empty.
4. **Model:** start with Random Forest / XGBoost; advanced option: LSTM.
5. **Split: time-based** train/validate/test split (never random) to avoid leakage.
6. **Evaluate:** MAE, RMSE, MAPE, R^2; compare against the division baseline.
7. **Deploy & retrain:** periodic retraining as new consumption data accumulates.

8.3 Final Prediction Formula

```
Time-to-Empty = (Current Volume - Minimum Safe Volume) / Predicted Flow Rate
```

Chapter 9 — Control Strategy, Safety & Security

9.1 Control State Machine

State	Condition (example)	Action
NORMAL	Tank > 60%	Valve 100%
WARNING	Predicted empty < 120 min	Alert; prepare rationing
RATIONING	Predicted empty < 60 min	Reduce valve to allowed flow
CRITICAL	Tank < 10%	Close valve; stop pump
RECOVERY	Refilled > 30%	Resume normal supply

9.2 Rationing Formula

```
Allowed Flow = (Current Volume - Min Safe Volume) / Target Survival Time
Valve Set-point = clamp(Allowed Flow / Normal Flow, min%, max%)

Example: usable 300 L, target 120 min --> allowed 2.5 L/min
(normal demand 10 L/min --> valve restricted accordingly)
```

Hysteresis is applied on all thresholds to prevent rapid state oscillation.

9.3 Edge Fallback (Cloud-Independent Safety)

```
if (level_percent < 10) { closeValve(); stopPump(); }
if (mqtt_down > 5 min && level_percent < 20) { setValve(30%); }
```

9.4 Security & Reliability

- MQTT over TLS with username/password or certificates; separate credentials for device and backend.
- API authentication (keys/JWT); firewall-restricted ports; HTTPS if exposed.
- Local buffering, watchdog timer, heartbeat topic, sensor-health detection with last-safe-state behavior.

Chapter 10 — Implementation Methodology (Phase Plan)

Phase	Key Tasks	Deliverable
1. Requirements	Problem, users, tank size, thresholds, component list	Specification & block diagram
2. Design	HW/SW architecture, MQTT topics, data schema, states	Design document
3. Hardware build	Prototype plumbing, wiring, float switch, power	Working prototype
4. Calibration	Level–volume map, pulses-per-liter, filtering	Stable, accurate readings
5. Firmware	Sensing, publishing, commands, actuation, failsafe	Functional edge node
6. Cloud setup	Broker, InfluxDB, ingestion, Grafana	Live dashboard + history
7. Baseline	Rule-based division predictor + alerts	Working non-ML baseline
8. Data collection	Varied usage profiles; synthetic augmentation if needed	Training dataset
9. ML development	Clean, feature-engineer, train, validate, compare	Trained model + evaluation
10. Integration	ML → decision engine → MQTT command → valve	Closed-loop smart system
11. Testing	Accuracy, latency, failure drills (see Ch. 11)	Test report & metrics
12. Documentation	Report, demo video, PPT, dashboard screenshots	Final submission package

Chapter 11 — Testing & Validation Plan

Test	Method	Pass Criteria
Level accuracy	Compare sensor vs. measured heights	Error < 2–3%
Flow accuracy	Timed collection into measuring container	Error < 5%
Prediction accuracy	Predicted vs. actual empty time	MAE < 15 min or MAPE < 20%
End-to-end latency	Telemetry→DB write; command→valve action	Command action < 3–5 s (local net)
Internet failure	Disable Wi-Fi; observe edge behavior	Local failsafe active; buffered replay works
Cloud failure	Stop backend; observe device	Device remains safe autonomously
Low-level safety	Drain to minimum	Pump stops; valve safe state
Rationing effectiveness	Same start volume, control ON vs. OFF	Controlled run extends availability measurably

Chapter 12 — Dashboard & Visualization Design

The Grafana dashboard presents, at a glance:

1. Tank level gauge (%) and current volume (L)
2. Live flow rate (L/min)
3. **Predicted time-to-empty countdown** (the headline panel)
4. Valve position and pump state
5. System state banner: NORMAL / WARNING / RATIONING / CRITICAL
6. Historical level and flow trend graphs
7. Prediction vs. actual comparison panel
8. Alert and event log

> Demo tip: style the dashboard like a control-room console; the countdown panel is the single most persuasive visual for evaluators.

Chapter 13 — Project Timeline (10 Weeks)

Week	Milestone
1	Requirements finalized; components procured
2	Hardware assembly & wiring complete
3	Calibration + ESP32 baseline firmware
4	MQTT + cloud ingestion running
5	InfluxDB + Grafana live dashboard
6	Rule-based prediction + alerts
7	Dataset creation + ML training
8	ML integrated with control loop
9	Full testing & validation (Ch. 11)
10	Report, presentation, demo video

Chapter 14 — Challenges & Mitigations

Challenge	Mitigation
Ultrasonic ripple noise	Stilling pipe + median/moving-average filtering + outlier rejection
Flow sensor error (air bubbles)	Calibration against measured volumes; multi-rate verification
Internet failure	Local buffering + replay; retained last valve state; edge failsafe rules
Inaccurate early predictions	Rule-based baseline fallback; periodic retraining; confidence thresholds
Valve/relay response delay	Lightweight MQTT; measured actuation timing kept under a few seconds
Power failure	UPS/battery for ESP32, router, relay; float-switch hard cut-off
Pump dry-run risk	Refill-only pump topology + mechanical float interlock

Chapter 15 — Expected Outcomes & Conclusion

15.1 Expected Outcomes

1. Real-time monitoring of tank level and consumption.
2. Accurate, explainable time-to-depletion prediction.
3. Proactive alerts before exhaustion occurs.
4. Automatic rationing that measurably extends water availability.
5. Safe autonomous behavior during network/cloud/power failures.

15.2 Conclusion

The project demonstrates a complete, production-style IoT pipeline — embedded sensing, MQTT networking, cloud time-series storage, ML prediction, visualization, and closed-loop actuation — applied to a socially relevant problem. By shifting water management from *reactive* (act after the tank is empty) to *predictive* (act before it empties), the system delivers tangible value while showcasing hardware, software, cloud, and data-engineering competence in a single coherent build.

Chapter 16 — Future Enhancements

- Multi-tank and building-wide optimization.
- Leak detection (level derivative vs. flow mismatch).
- Mobile app with push notifications.
- Weather/occupancy data as additional ML features.
- Automated tanker ordering integration at CRITICAL state.
- Federated learning across multiple installations.

Appendix A — Professional Phrasing Guide

Avoid Saying	Say Instead
“The AI divides volume by flow rate.”	“The model predicts future demand from historical consumption patterns; time-to-depletion is derived from predicted demand and usable volume.”
“The solenoid valve partially closes.”	“A motorized ball valve adjusts flow percentage; with a binary solenoid, timed duty-cycle rationing is used.”
“The cloud controls everything.”	“Cloud performs prediction and coordination; the edge enforces local failsafes, ensuring safe operation without connectivity.”

Appendix B — MVP vs. Advanced Scope

MVP (guaranteed solid pass)	Advanced (distinction grade)
Level + flow telemetry → MQTT → InfluxDB → Grafana; rule-based prediction; alerts; manual/semi-auto valve	ML demand forecasting; automatic percentage valve control; closed cloud–edge loop; Docker deployment; MQTT TLS; edge fallback intelligence; usage-pattern analytics

End of document. Replace bracketed placeholders on the cover page before submission.